Solution: Buddy Strings

Problem Statement

Given two strings `a` and `b`, return `true` if they can be considered "buddy strings". Otherwise, return `false`.

Two strings are `buddy strings` if you can `swap` any two letters in one string to make it `equal` to the other string.

Examples

- **Example 1**:
  - **Input**: `a = "xy", b = "yx"`
  - **Expected Output**: true
  - **Justification**: Swapping 'x' and 'y' in string `a` results in "yx", which matches string `b`.
- **Example 2**:
  - **Input**: `a = "ab", b = "cd"`
  - **Expected Output**: false
  - **Justification**: There are differences at every position, and swapping any characters in `a` cannot make it equal to `b`.
- **Example 3**:
  - **Input**: `a = "abcde", b = "acbde"`
  - **Expected Output**: true
  - **Justification**: Swapping 'b' and 'c' in string `a` results in "acbde", which matches string `b`. So, both strings are buddy strings.

Solution

To solve this problem, we start by comparing the lengths of the two strings. If their lengths differ, they cannot be buddy strings. For strings of equal length, we track the positions where they differ. If there are no differences and the first string has at least one repeating character, it means a swap could occur within the string to make it its own buddy string. If there are exactly two differences, we check if swapping the characters at these positions in one string makes the two strings equal. This approach ensures we accurately identify buddy strings by addressing both scenarios where strings are initially equal or need a specific swap to become equal.

Our approach is effective because it minimizes the number of checks needed to determine if two strings are buddy strings. By first checking the length, then the positions of difference, and finally the conditions for being buddy strings, we efficiently narrow down the possibilities with minimal computational overhead. This method balances thoroughness with efficiency, making it suitable for a wide range of inputs.

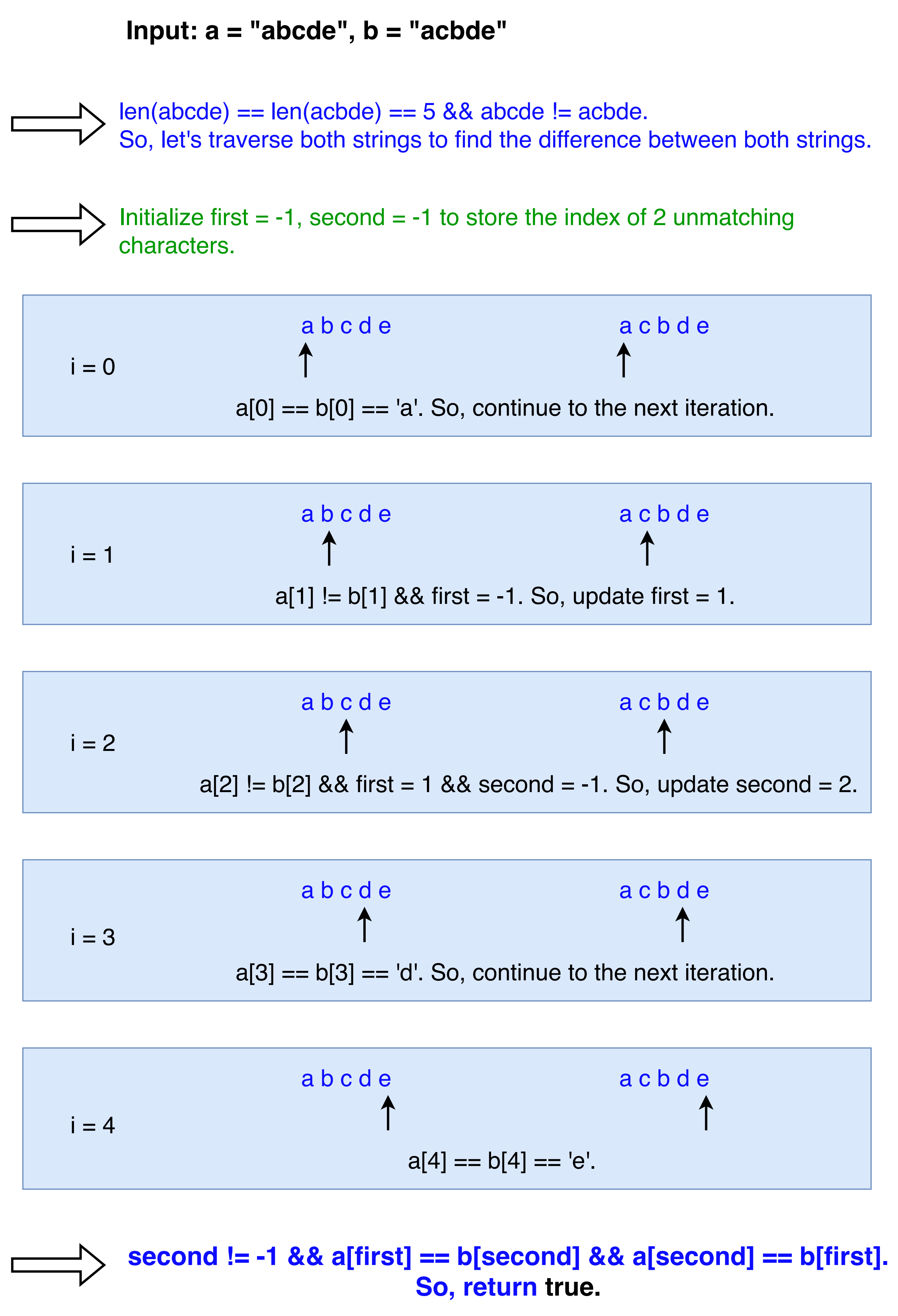
Step-by-step Algorithm

1. **Initial Length Check**:
   - If `a.length` is not equal to `b.length`, return false immediately, as strings of different lengths cannot be buddy strings.
2. **Handle Identical Strings**:
   - If the strings `a` and `b` are identical, then for them to be buddy strings, there must be at least one character in `a` that appears more than once. Use a frequency count array or a set to check for repeating characters. If a repeat is found, return true.
3. **Identify and Record Differences**:
   - If `a` and `b` are not identical, iterate through both strings simultaneously to compare characters at each position.
   - Record the indices where `a` and `b` differ. Use two variables, `first` and `second`, initialized to -1, to store these indices.
   - If more than two differences are found, return false as a swap cannot resolve more than two differences.
4. **Check for Exactly Two Differences**:
   - After identifying the indices of differences, check if swapping the characters at these positions in `a` would make it equal to `b`. This means that the character at `first` index in `a` should match the character at `second` index in `b` and vice versa.
5. **Final Verification**:
   - If the conditions in step 4 are satisfied, return true, indicating that the strings are buddy strings by a single swap.
   - Otherwise, return false.

Algorithm Walkthrough

Consider the input `a = "abcde", b = "acbde"`.

1. **Initial Length Check**:
   - Both strings `a` and `b` are of the same length. This check passes, allowing us to proceed to the next step.
2. **Handle Identical Strings**:
   - The strings `a` and `b` are not identical, so we move to the next step. There's no need to check for repeating characters since the strings differ.
3. **Identify and Record Differences**:
   - As we iterate through both strings from the beginning, they match at the first position. At the second position, we encounter a difference ('b' in `a` vs 'c' in `b`), so we record `first = 1`.
   - Continuing, we find another difference at the third position ('c' in `a` vs 'b' in `b`), so we record `second = 2`. The rest of the characters match in both strings.
   - There are exactly two differences, and no more, which is what we need for a potential swap to make `a` and `b` equal.
4. **Check for Exactly Two Differences**:
   - With `first = 1` and `second = 2`, we check if swapping the characters at these positions in `a` would make it equal to `b`. Specifically, we're looking to see if after the swap, `a[first]` equals `b[second]` and `a[second]` equals `b[first]`.
   - Before the swap: `a = "abcde"`, `b = "acbde"`
   - After a hypothetical swap in `a` (swapping 'b' and 'c'): `a` would become "acbde".
   - This swapped version of `a` matches `b` exactly.
5. **Final Verification**:
   - Since swapping 'b' and 'c' in `a` makes it identical to `b`, the strings are buddy strings. Therefore, we return **true**.



Input: a = "abcde", b = "acbde"

len(abcde) == len(acbde) == 5 && abcde != acbde.
So, let's traverse both strings to find the difference between both strings.

Initialize first = -1, second = -1 to store the index of 2 unmatching characters.

a[0] == b[0] == 'a'. So, continue to the next iteration.

a[1] != b[1] && first = -1. So, update first = 1.

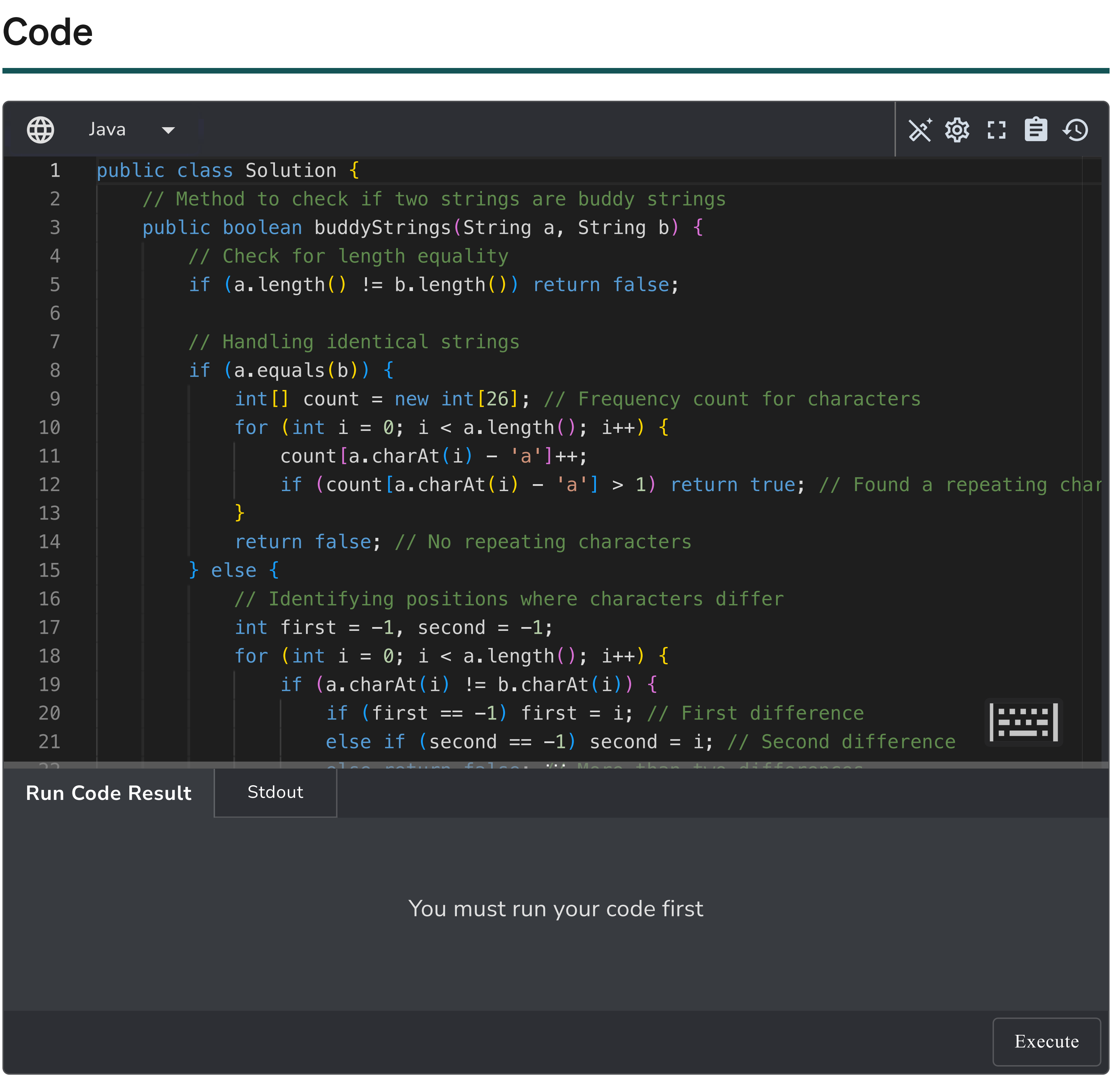
a[2] != b[2] && first = 1 && second = -1. So, update second = 2.

a[3] == b[3] == 'd'. So, continue to the next iteration.

a[4] == b[4] == 'e'.

second != -1 && a[first] == b[second] && a[second] == b[first].
So, return true.

Code



```java
public class Solution {
    // Method to check if two strings are buddy strings
    public boolean buddyStrings(String a, String b) {
        // Check for length equality
        if (a.length() != b.length()) return false;

        // Handling identical strings
        if (a.equals(b)) {
            int[] count = new int[26]; // Frequency count for characters
            for (int i = 0; i < a.length(); i++) {
                count[a.charAt(i) - 'a']++;
                if (count[a.charAt(i) - 'a'] > 1) return true; // Found a repeating char
            }
            return false; // No repeating characters
        } else {
            // Identifying positions where characters differ
            int first = -1, second = -1;
            for (int i = 0; i < a.length(); i++) {
                if (a.charAt(i) != b.charAt(i)) {
                    if (first == -1) first = i; // First difference
                    else if (second == -1) second = i; // Second difference
```

You must run your code first

Complexity Analysis

- **Time Complexity**: $O(n)$ where n is the length of the strings, as the algorithm iterates over the strings at most once.
- **Space Complexity**: $O(1)$ for checking differences and $O(n)$ in the worst case when the strings are identical to store character frequencies. However, because the character set is limited (assuming ASCII), this can also be considered $O(1)$ in practical scenarios where the alphabet size is fixed.

← Previous
Buddy Strings (Easy)

Next →
Can Place Flowers (Easy)

☐ Mark as Completed